

ThinStation-NG 7.2

Compilación de una imagen universal en un equipo Fedora
Sesión RDP en kiosk · acceso SSH · máxima compatibilidad de hardware

Contenido verificado contra el repositorio oficial (rama 7.2-Stable) · Mayo 2026

Repositorio	github.com/thinstation/thinstation-ng · rama 7.2-Stable
Entorno	Un equipo Fedora (físico o VM), trabajando como root . Otras distros requieren VM Fedora.
Compatibilidad	Imagen universal : todos los módulos de kernel, todos los drivers de vídeo abiertos y todo el firmware
Sesión	RDP con diálogo de IP nativo del paquete <i>freerdp</i> (sin scripts propios)
Acceso remoto	Servidor SSH (dropbear) incluido en la imagen para administración y diagnóstico

1. Visión general

ThinStation-NG es un sistema operativo de cliente ligero de código abierto. Este tutorial genera una imagen **ISO + PXE** pensada para arrancar en el mayor número posible de equipos, con una sesión RDP en modo kiosco que pide la IP del servidor al arrancar. Todo el trabajo se hace por línea de comandos en un equipo Fedora; el entorno gráfico vive dentro de la imagen, no en la máquina que compila.

El proceso tiene dos fases:

Fase	Herramienta	Duración	Repite al recompilar
Poblar el chroot (Fedora 42 + herramientas TS)	<code>./setup-chroot</code>	~25–40 min	No — queda en disco
Compilar la imagen (paquetes + kernel + squashfs + ISO)	<code>./build</code>	~20–40 min	Sí — usa caché

Ubicación de la ISO y el PXE

Tras compilar, los artefactos quedan en `<repo>/ts/build/boot-images/grub/`. GRUB es el único método de arranque no obsoleto en la 7.2.

2. Entorno requerido: Fedora

La compilación **solo funciona en Fedora**, por dos motivos verificados en el propio código:

- **El chroot se construye con dnf.** El script usa `dnf install --installroot ... --releasever=42` para descargar la base del sistema desde los repositorios de Fedora. Otras distribuciones no tienen `dnf` ni esos repos, así que esta fase falla.
- **El empaquetado final necesita root real.** El último paso, que crea la imagen de arranque EFI, usa dispositivos `loop` y `mount`, que requieren privilegios reales sobre el kernel.

Si NO usas Fedora

Si trabajas en Ubuntu, Debian, Mint u otra distribución, **instala una máquina virtual con Fedora** (edición Server, sin escritorio: basta línea de comandos) y realiza todo el proceso dentro de ella. Intentar compilar fuera de Fedora hará que el proceso falle.

En adelante, «el equipo Fedora» se refiere indistintamente a una instalación física de Fedora o a esa VM Fedora. Todos los comandos se ejecutan como root.

3. Requisitos previos

Según el README oficial, el host Fedora necesita:

- **8 GB de RAM** — Recomendado; con 4 GB irá lento.
- **40-50 GB de disco libre** — El README pide 30, pero una imagen universal (todos los módulos y firmware) ocupa bastante más durante el build.
- **Privilegios de root** — Trabaja como root, por ejemplo con `sudo -i`.
- **Conexión a internet** — El chroot descarga la base de Fedora 42 y los paquetes.

4. Guía paso a paso

Paso 1 Instalar dependencias

En el equipo Fedora, como root, instala las utilidades necesarias. El README lista `dnf chroot git`; conviene añadir `util-linux`, `which` y `dosfstools`. El resto (`mksquashfs`, `xorriso`, `grub2`) lo arrastra `setup-chroot`.

coreutils (que aporta 'chroot') ya viene en Fedora

```
sudo -i          # trabajar como root
dnf install -y git util-linux which dosfstools
```

Paso 2 Clonar el repositorio

Clona la rama **7.2-Stable** (es una rama, no una etiqueta). El directorio resultante será la raíz del chroot, en una partición con espacio suficiente.

Working dir = /root/thinstation-ng

```
cd /root
git clone --branch 7.2-Stable \
  https://github.com/thinstation/thinstation-ng.git
cd thinstation-ng
```

Paso 3 Poblar el chroot

`./setup-chroot` descarga la base de Fedora 42, instala los paquetes del sistema, monta `/dev`, `/proc` y `/sys` dentro del chroot y abre su shell. **Tarda ~25-40 min la primera vez.**

Desde /root/thinstation-ng

```
./setup-chroot
```

Al terminar verás el prompt del chroot: `[root@TS_chroot]/#`. Es normal ver algunos avisos cosméticos (paginadores, `systemd/dconf` ejecutándose dentro de un chroot); no impiden la compilación.

Paso 4 Ir al directorio de build

Dentro del chroot, `/build` es un enlace a `/ts/build`, donde están `build.conf`, `thinstation.conf.buildtime` y `./build`.

Dentro del chroot

```
cd /build
```

Paso 5 Configurar build.conf (paquetes)

Activa (descomenta) estas líneas. Los nombres están verificados contra 7.2-Stable. Para máxima compatibilidad se incluyen los **tres drivers de vídeo abiertos** (Intel, AMD y Nvidia) además de los de VM, y el servidor sshd para administración remota:

Línea a activar	Función
package xorg7	Servidor X con driver universal <i>modesetting</i> (KMS)
package xorg7-intel	Driver de GPU Intel
package xorg7-amdgpu	Driver de GPU AMD/ATI
package xorg7-nouveau	Driver de GPU Nvidia (abierto)
package xorg7-pxl / xorg7-vmware	Drivers de GPU para máquinas virtuales
package autonet	Red automática (DHCP)
package freerdp	Cliente RDP + diálogo de IP + reconexión
package locale-es_ES	Teclado y locale en español
package sshd	Servidor SSH (dropbear) para acceso remoto

Nombres que NO existen en 7.2-Stable (causan error)

xorg → xorg7 · netbase → autonet · xorg-video-vesa → no existe (KMS va en xorg7) ·
dialog/xterm → no son paquetes TS-NG.

El firmware se incluye por completo de serie: param `allfirmware true` ya es el valor por defecto, así que no tienes que tocar nada para cubrir Wi-Fi, Bluetooth y firmware de GPU.

Paso 6 Configurar thinstation.conf.buildtime

La clave: el paquete freerdp incluye `/etc/cmd/freerdp.getip`, que dispara el diálogo de IP cuando no hay servidor fijo. **Sin scripts propios**. Sustituye la línea por defecto `SESSION_0_TYPE=xfwm4` y deja:

`/build/thinstation.conf.buildtime`

```
# Sesión RDP en modo kiosco
SESSION_0_TYPE=freerdp
SESSION_0_AUTOSTART=on
# NO poner SESSION_0_FREERDP_SERVER -> dispara el dialogo de IP
SESSION_0_FREERDP_OPTIONS="/multimon /dynamic-resolution +clipboard /network:auto"
FREERDP_CERTIGNORE=on      # equivale a /cert:ignore
RECONNECT_PROMPT=on        # al cerrar, vuelve al dialogo de IP

# Red / zona horaria / idioma
NET_USE=BOTH
NET_USE_DHCP=on
TIME_ZONE=Europe/Madrid
LOCALE=es_ES
```

Teclado (KEYMAP)

KEYMAP=es NO existe en 7.2. El teclado español se activa con `package locale-es_ES + LOCALE=es_ES`.

Paso 7 Compilar la imagen universal

Compila con `--allmodules`, que incluye **todos** los módulos del kernel (redes, almacenamiento y, sobre todo, los DRM/KMS de cualquier GPU). Esto es lo que hace la imagen universal; ver la sección 5. Las otras dos flags evitan las confirmaciones (aceptan licencias y descargan binarios sin preguntar) y se guarda un log:

En /build, dentro del chroot · ~20-40 min

```
./build --allmodules --license ACCEPT --autodl 2>&1 | tee /build/build.log
```

Si se interrumpe, relanza el mismo comando (usa caché). Al terminar, verifica que la ISO existe y no quedó en 0 bytes:

Debe mostrar varios cientos de MB

```
ls -lh /build/boot-images/grub/thinstation-efi.iso
```

Para sacar la ISO del equipo Fedora a otra máquina (o a un USB), cópiala desde su ruta: `/root/thinstation-ng/ts/build/boot-images/grub/thinstation-efi.iso`. Si compilas en una VM, puedes traerla con `scp`.

5. Por qué la imagen es universal

El objetivo es una sola ISO que arranque en el mayor número de equipos posible, sin afinarla por modelo. Eso se consigue combinando tres elementos, todos ya aplicados en este tutorial:

Elemento	Qué aporta	Cómo se activa
Todos los módulos de kernel	Drivers de red, almacenamiento y vídeo (DRM/KMS) para casi cualquier hardware	<code>./build</code> <code>--allmodules</code>
Driver X universal + específicos	<i>modesetting</i> (en xorg7) funciona con cualquier GPU que tenga KMS; los drivers Intel/AMD/Nvidia complementan	Paquetes <code>xorg7</code> , <code>xorg7-intel/amdgpu/nouveau</code>
Todo el firmware	Wi-Fi, Bluetooth y firmware de GPU sin tener que listarlos	<code>param allfirmware true</code> (por defecto)

La pieza más importante es `--allmodules`: garantiza que el módulo KMS de la GPU del equipo (i915 de Intel, `amdgpu` de AMD, `nouveau` de Nvidia, etc.) esté presente, de modo que el driver *modesetting* de Xorg pueda inicializar la pantalla. Esa es justo la diferencia entre arrancar gráfico en una VM y no hacerlo en un equipo físico.

La contrapartida: tamaño

Incluir todos los módulos, drivers y firmware hace la imagen bastante más grande (y un arranque ligeramente más lento) que una ajustada a un modelo concreto. Es el precio de la universalidad, y para un parque heterogéneo casi siempre compensa: una sola imagen para todos los equipos.

No necesitas perfiles de máquina (`machine . . .`) ni capturas por modelo: con `--allmodules` la imagen ya lleva lo de todos. Las líneas `machine` que vengan por defecto en `build.conf` son inocuas y puedes dejarlas.

6. Acceso y depuración por SSH

El paquete `sshd` (`dropbear`) incluido en el Paso 5 permite administrar el cliente y sacar logs cómodamente. La imagen es de solo lectura, así que SSH se incluye **al compilar**, no se instala sobre el cliente arrancado. El paquete trae `systemctl enable dropbear`, así que **arranca solo**; y su `ExecStart` no lleva la opción `-w`, por lo que **permite login de root**.

Credenciales por defecto

Usuario `root`, contraseña `pleasechangeme` (de `param rootpasswd` en `build.conf`); el usuario normal es `tsuser` con la misma contraseña. Cámbialas en `build.conf` (`param rootpasswd / tsuserpasswd`) antes de cualquier despliegue real.

Tras arrancar el cliente, conéctate desde otra máquina (la IP la tienes en tu DHCP/router, o con `ip a` en la consola del cliente):

Logs por usuario también en `/run/user//applications/` (`session.*` y `xorg.*`)

```
ssh root@IP_DEL_THINCLIENT
```

```
# Logs utiles (se pueden copiar con scp):
```

```
cat /var/log/Xorg.0.log
```

```
# log del servidor X
```

```
dmesg | grep -iE 'drm|i915|amdgpu|radeon|nouveau'
```

```
# driver KMS cargado
```

```
lspci | grep -iE 'vga|display|3d'
```

```
# GPU del equipo
```

```
cat /var/log/boot.log ; cat /var/log/messages.log
```

```
# arranque y syslog
```

Seguridad

La clave de host de dropbear viene fijada en el repositorio (igual para todos). Para depurar es indiferente, pero en producción conviene regenerarla y no dejar el login de root con la contraseña por defecto, sobre todo si el equipo queda expuesto a la red.

7. Recompilación posterior

El chroot persiste en el directorio del repositorio. Para recompilar tras un cambio:

Recompilación completa (mantén --allmodules para conservar la imagen universal)

```
cd /root/thinstation-ng
./setup-chroot      # detecta que ya esta instalado y entra directo
cd /build && ./build --allmodules --license ACCEPT --autodl
```

Si solo cambiaste `thinstation.conf.buildtime` (sin tocar `build.conf`), el build solo regenera las imágenes de arranque, que es lo más rápido.

8. Mecanismo del diálogo de IP nativo

El «pedir IP al arrancar y reconectar al cerrar» no requiere scripts propios; está en el paquete `freerdp`:

Elemento	Qué hace
<code>freerdp.getip</code>	Su presencia activa <code>CMD_GETIP=true</code> (incluido por defecto).
<code>read_options()</code>	Con <code>CMD_GETIP</code> y <code>\$SERVER</code> vacío, pone <code>ALLOW_SERVER_EDITS=true</code> .
<code>dialog_get_server_address()</code>	Muestra un diálogo GTK editable para teclear la IP.
<code>xfreerdp</code>	Se lanza con la IP y las opciones de <code>SESSION_0_FREERDP_OPTIONS</code> .
<code>check_reconnect()</code>	Al cerrar la sesión, con <code>RECONNECT_PROMPT=On</code> vuelve al diálogo.

9. Resolución de problemas

Aun con --allmodules, un equipo se queda en login de consola sin diálogo

Mensaje «not able to go graphical»: Xorg no levantó. Con la imagen universal es poco frecuente, pero revisalo por SSH: `cat /var/log/Xorg.0.log` (no `screens found`) y `dmesg | grep -i drm` (¿ligó un driver KMS?). Comparte esa salida para diagnosticar la GPU concreta.

La ISO no se genera / queda en 0 bytes; xorriso: 'Boot image file is empty'

El paso EFI no pudo usar `loop/mount`: no estás en un entorno con root real. Compila en Fedora directamente o en una VM Fedora (sección 2), no en entornos restringidos.

La fase setup-chroot falla al descargar la base del sistema

Casi siempre es que no estás en Fedora (no hay `dnf` ni repos compatibles). Usa un equipo o VM Fedora (sección 2).

build.conf con nombres de paquete incorrectos

Error: package X does not exist. Usa xorg7 y autonet; dialog/xterm no son paquetes TS-NG.

El build se para pidiendo confirmación

Usa siempre `./build --allmodules --license ACCEPT --autodl`.

No aparece el diálogo de IP (con X funcionando)

Probablemente quedó `SESSION_0_FREERDP_SERVER` en buildtime, o el tipo de sesión no es freerdp.

No puedo entrar por SSH al cliente

Confirma package `sshd`, que el cliente tiene IP (`ip a`) y usa `root / pleasechangeme` (o tu `rootpasswd`).

10. Referencia rápida de comandos

Dónde	Comando	Propósito
Fedora (root)	<code>dnf install -y git util-linux which dosfstools</code>	Dependencias
Fedora (root)	<code>git clone --branch 7.2-Stable https://github.com/thinstation/thinstation-ng.git</code>	Clonar repo
Fedora (root)	<code>cd thinstation-ng && ./setup-chroot</code>	Poblar / entrar al chroot
Chroot	<code>cd /build && ./build --allmodules --license ACCEPT --autodl 2>&1 tee /build/build.log</code>	Compilar imagen universal
Chroot	<code>ls -lh /build/boot-images/grub/thinstation-efi.iso</code>	Verificar la ISO
Cliente	<code>ssh root@IP_THINCLIENT · cat /var/log/Xorg.0.log · dmesg grep -i drm</code>	Acceso y diagnóstico

Tutorial generado en mayo 2026 · Verificado contra el repositorio oficial `thinstation/thinstation-ng` rama 7.2-Stable, e incorporando las lecciones de la puesta en marcha real (host Fedora, imagen universal de máximo hardware y acceso SSH).